

高级语言程序设计：期末复习

考核方式与核心知识点回顾

史峰源 人工智能学院
南开大学

2026 年 1 月 2 日

目录

- 1 讲座结构与考核方式
- 2 基础知识
- 3 往年笔试真题回顾
- 4 C++ 上机题
- 5 More to Explore

讲座结构与考核方式

概览

- C++ 概述
- C++ 考核方式
- C++ 笔试知识点概述（按章节顺序）
- C++ 往年题回顾
- C++ 机试（2024 年真题及建议）
- Q&A

考核方式

总成绩构成：

- 笔试：50%
- 上机考试：30%
- 平时分：20%（OJ 作业需全部 AC）

笔试题型分布

- ① 选择题： 10×2 分
- ② 改错题： 8×2 分
- ③ 读程序写结果： 6×4 分
- ④ 程序填空： 10×2 分
- ⑤ 程序设计： 2×10 分

注：知识点覆盖广，复习难度较大。

机试

- 一般 3-4 道题，全部 AC 即可满分。
- 附加题策略：前面题目全部完成后，写出附加题可加总成绩 5 分。
- 难度：与平时上机练习相近，部分题目可能略简单。

基础知识

数制转换

● 数制转换：二进制、八进制、十进制、十六进制。

计算机与程序设计语言 ☐
数制转换与数据存储 ☒
C++语言的基本概念 ☐
C++语言的词汇 ☐

☐ 数的进制
☒ 数制转换
☐ 数据存储



数制转换



补码运算

十进制数转换为二进制数

计算机与程序设计语言
数制转换与数据存储
C++语言的基本概念
C++语言的词汇

☐ 数的进制
☒ 数制转换
☐ 数据存储



数制转换

十进制负数转换为二进制数

- 将十进制负数-22转换为二进制数
 - 将22转换为二进制数
 - 10110
 - 补足4个字节的二进制数位（根据系统中整数存储的位数）
 - 00000000 00000000 00000000 00010110
 - 按位取反
 - 11111111 11111111 11111111 11101001
 - 加1
 - 11111111 11111111 11111111 11101001+1
 - =11111111 11111111 11111111 11101010

允公允能 日新月异

计算机学院&网络空间安全学院

21 年程序设计题：将 32 位的二进制数转换成补码形式输出

关键字以及标识符

- 关键字：void int if else for while

- 以字母或下划线 “_” 开头，由字母、数字、下划线组成的字符串
- 不能与关键字重名
- 标识符区分大小写
- 有效长度有规定

- 标识符：命名规则

字面值常量

- 整型/字符型/字符串/布尔型/指针 (nullptr)/0/用户自定义
- 输出常考点：自动转换为十进制输出
 - 'cout << 023 << 23 << 0x23<<0b10;'
 - 19 (八进制), 23 (十进制), 35 (十六进制), 2(二进制)

运算符

计算机与程序设计语言 ☐
 数制转换与数据存储 ☐
 C++语言的基本概念 ☐
 C++语言的词汇 ☒

☐ 关键字
☐ 标识符
☐ 字面值常量
☒ 运算符

☐ 分隔符



运算符 (operator)

由字母、数字之外的第三类基本符号组成
 个别关键字如sizeof、new、delete，也被认为是运算符

其余运算符为：

+, -, *, /, %, =, !=, <, <=, >, >=
 =, !, &&, ||, &, ^, |, ~, ++, --, +=, -=, *=, /=, %=
 =, <<=, >>=, &=, ^=, |=, ?:, =, (), [], .., ->,
 <<, >>, ,, ::

- 某些运算符还有其它用途，如乘运算符*

数据类型概览

- **常量**：字面值, const, define。
- **变量修饰符**： static, thread-local, extern, mutable。
- **整型/浮点型** short, signed, float, double.....
- **字符型**： char, signed/unsigned char (需记住 A, a 的 ASCII 码)
- **布尔型**：非零即 true, 0 为 false。
- **无值型**：函数的参数, 指针, 声明函数的返回值

枚举 (Enum)

枚举值本质是整型常量。

```
enum color RED=1, YELLOW, BLUE c1=BLUE;  
// RED=1, YELLOW=2, BLUE=3  
enum day Sun, Mon, Tue;  
// Sun=0 (默认从 0 开始)
```

类型转换与说明符

- **显式转换**: `static_cast<int>(d1+d2)`
- **CV 限定符**: `const, volatile`
- **说明语句**:
 - `auto` 推导
 - `using BigOnes = unsigned long long;`
 - `typedef int id_number;`

运算操作

- 赋值与运算操作: demo1
- 优先级和结合性: 算术运算符, 关系运算符, 双目逻辑运算符的优先级均高于赋值运算符; 赋值运算符的结合性是自右向左
($k=i=j+5, k=(i=j+5)$)
- 隐式类型转换: $10+'a'+1.5-2.25* 'b'$
- 自增/自减: $++i, i++, -i, i-$ (int a=3; a+=a++*a; a=13)

逻辑短路问题

读程序写结果易考点：

- **逻辑与 (&&)**：第一个为 false，后面不执行。
- **逻辑或 (||)**：第一个为 true，后面不执行。

```
int i=3, j=5;
```

```
int c = i > j  j++ || i++;
```

```
// 结果: i 变为 4, j 保持 5, c 为 1
```

其它运算

- 位运算：& | << >>
- 三目运算符：三目运算符 ? : int n = ++i == 0 ? i == -1 ? 11:22;
- 逗号运算符：逗号表达式的值是最右边的值

语句与流程控制

- 语句 if switch while for
- 闰年判断: `'year%4==0 && year%100!=0 || year%400==0'`
- Switch: 注意 'break' 和 'default' 的作用。
- 循环: do-while 与 while 的区别。
- 转向语句: break, continue, goto, return

数组核心

- 数组的内存空间是连续的。(重要!)
- 数组的初始化 `int B [3]=4,3,2;int B [10]=4,3,2;int B []=4,3,2`
- 二维数组存储: 转换成一维方式存储在内存 ($L \times N1 \times N2$)。
- 字符数组初始化:
 - 'char s[] = "china";' (含 " , 长度 6)
 - 'char s[] = 'c','h';' (不含 " , 长度 2)

Struct vs Union

结构体 (Struct)

- 可以直接对结构体变量名赋值 `stu1=stu2`

上机考试必考内容。

联合体 (Union)

- 不同类型的变量共用同一段内存空间。
- 不允许直接初始化。
- 不能直接对联合变量名赋值 (`'stu1 = stu2'` 错误)。



函数注意事项

- **声明与定义**：声明必须在调用前；声明可省略参数名，定义不可。
- **内联函数 (inline)**：建议编译器内联展开。
- **参数传递**：
 - 数组做参数：传递的是首地址（引用传递效果）。
 - `'void swap(int a, int b)'`
- **形参与实参**
- **函数的递归**：画图理解，读程序写结果会涉及到

重载与静态变量

- **函数重载**：关键在于能否将函数区分开！
 - `printStar()` `printStar(int)` `printStar(int ,int)`
 - `int abs(int n)` `float abs(float f)`
 - `int get(int n, float a [])` `int get(int n, float a[], int c=5)`
 - `void print(double)` `void print(double )`
- **静态局部变量 (static)**：
 - 局部作用域，全局生存期。
 - 具有记忆性，未初始化时自动为 0。
 - **读程序写结果常考！**
- **运算符重载**：同样从函数的角度去理解
 - `string operator -(string, string)`
 - `string operator +(string, string)`
 - `operator+(b1,b2)` `b1+b2`

指针概念

- 指针是一个存储内存地址的整数。

指针运算 (重点)

- * 解引用, 取内容运算 (开锁)
- & 取地址运算 (找门牌号)
- 指针的算术运算 (+, -, ++, --)
- 数组与指针之间的运算: 数组的名字就是一个地址, 一个指针常量
 - `int a[10], int * q`
 - `a[0] * a`
 - `a[9] * (a+9)`
 - `a[9] a+9`
 - `a+=3, a--` 错误!
- 指变量间的算术运算
 - 算术运算; `p-q, +` 没有意义
 - 关系运算

常量指针与指针常量 (重点)

- **指针常量 (`char * const ptr`)**: 地址不能变 (不能指向别的房子), 内容可以变。
 - `char * const ptr = ch`
 - `* ptr = 'b'`
 - `ptr = ch1` 错误
- **常量指针 (`const char * ptr`)**: 指向常量的指针。不能通过该指针修改内容 (常用于函数参数保护数据)。
 - `char ch = 'a', ch1 = 'x'`
 - `const char * ptr = ch;`
 - `* ptr = 'b'`
 - `ch = 'b', ptr = ch1`

高级指针 (重点)

- 数组指针: `'int (*pa)[4]'` (指向数组的指针)。
- 指针数组: `'int *pa[4]'` (存放指针的数组)。
- 多重指针
- 字符指针数组
- 指针参数:
 - 指针作为参数
 - 函数返回指针
 - 指向函数的指针
- 动态内存: `'new' / 'delete'`。
- 引用: 必须初始化, 作为别名使用。

往年笔试真题回顾

程序改错类型

- 少分号;
- 函数返回值类型不匹配, `void func()return 1`
- 部分分支没有返回值
- 数组改错 `int a[10] ;a=1,2,3`
- 访问数组越界
- 常变量改值 `const int a=5; a++`
- 运算符错误 `if(a=1)`
- 作用域错误 (两个文件的变量/函数作用域问题, `static` 和 `extern`)
- `if else` 不匹配问题
- 逻辑错误问题, `i+1` 写成 `i`
- `'0'<=str[i]<='9'`

读程序写结果

- 易考点

- 循环，判断语句
- 函数递归调用
- static 变量问题（24 年）
- 考虑到整数除法/类型转换问题 $n=n/2$

- 建议

- 尽量在草稿纸上手写，避免出错
- 看往年题与 PPT 真题

程序填空与设计

- **程序填空**注意读题干程序的功能要求看往年题与 PPT 上的程序填空题，难度不是太大
- **程序设计**
 - 两题，10 分
 - 第一题较为简单，基础的程序实现
 - 第二题较为开放，可能给某个背景下的程序实现，例如（24 年，判断美国大选票数是否造假）
 - 建议，理清思路再写代码，重点要展现自己的思路，不要纠结于细节
 - 多写注释，或使用注释/伪代码大法

C++ 上机题

真题回顾

● 2023 年

- 第一题：基本运算（求 10 个整数最大值最小值和，将字符串大写字母转换成小写字母）
- 第二题：矩阵运算（矩阵 A, B 中素数元素 +1, 求和, 奇数元素-1, 求差, 偶数元素 $\times 2$, 求积）
- 第三题：函数（重载函数, 实现不同数据之间的进制转换）
- 第四题：结构体（实现坐标点结构体, 直线结构体, 矩形结构体, 判断是否平行, 矩阵是否重叠）
- 附加题：二分

● 2024 年

- 第一题：进制转换（用重载函数实现八进制字符串与十进制数的转换）
- 第二题：结构体（实现结构体, 并根据结构体内的元素进行排序）
- 第三题：黑白格子（判断有无符合要求的子矩形）真题带看

More to Explore

More to Explore

- **Courses:** CS106B/X, CS106L
- **一些学习算法的网站:**
 - Ol-wiki
 - <https://labuladong.online/algo/>
 - 洛谷题单, 力扣
 - 牛客网
 - <https://github.com/TheAlgorithms/C-Plus-Plus>
 - Acwing 的算法课
- **推荐的书籍**
 - 深入浅出程序设计竞赛
 - C++ primer
 - 人间词话

Something Interesting

- github,awesome
- copilot
- Vibe coding
- csdiy.wiki
- <https://github.com/summerwind0131/NKU-AI-Study>
- <https://github.com/Luhaozhhhhhe>
- <https://github.com/NKU-CS-ICU/NKU-CS-ICU>
- <https://github.com/TephrocactusHC/NKU-CS-SAVE>

Q & A

Q & A

新年快乐！ 祝大家期末考试顺利！

上机前记得多刷 OJ，把 AC 过的代码再看一遍
笔试前多复习 PPT 知识点
不要焦虑，大家都是最棒的！